

Universidad Rafael Landívar

Facultad de Ingeniería

Licenciatura en Ingeniería en Informática y Sistemas

Manejo e Implementación de Archivos Sección: 2

Docente: Ing. Luis Rojas

PRÁCTICA 2, SEMANA 13

“API Secrets y Autenticación”

Estudiantes: Marco Fabrizzio Donadio Castellanos

Carné: 1076925

Guatemala 02 de octubre de 2026

Objetivo: Practicar el consumo de una API REST autenticada y aplicar una buena práctica básica de seguridad: evitar colocar credenciales directamente en el código fuente. El estudiante utilizará Postman, una aplicación C# y GitHub.

Parte 1 – Probar autenticación con Postman

Para esta práctica se utilizará DummyJSON, una API pública de pruebas que permite realizar un inicio de sesión y obtener un token de acceso.

Documentación: <https://dummyjson.com/docs/auth> <https://dummyjson.com/docs/users>

1. Crear la solicitud de inicio de sesión

a) Abra Postman y cree una nueva colección llamada: MIA – Semana 13.

b) Cree una solicitud llamada: Login.

c) Seleccione el método POST.

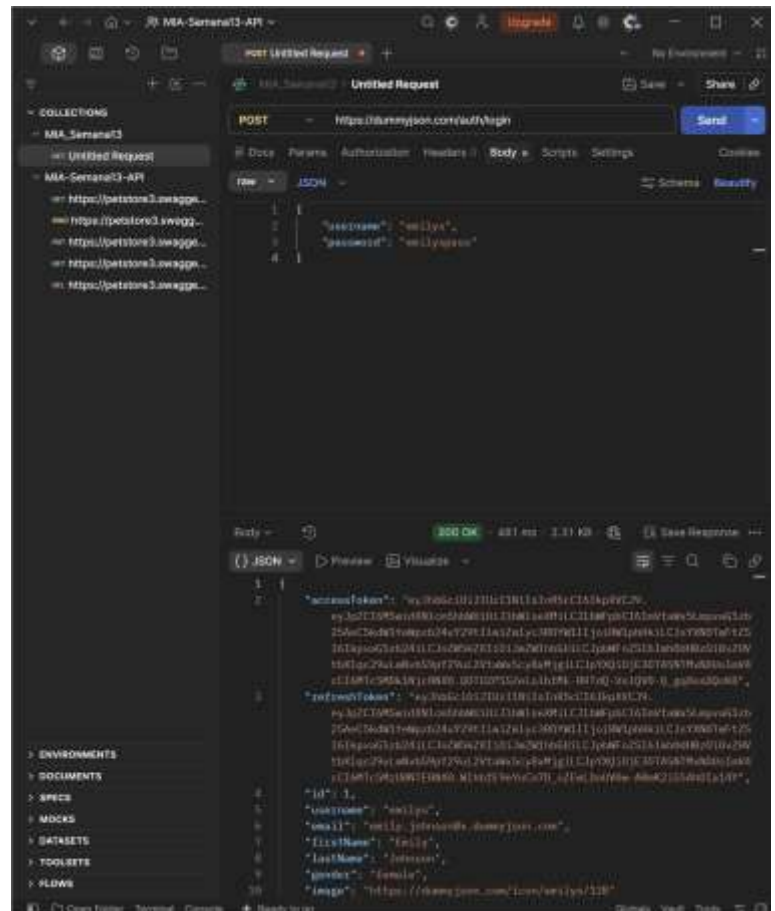
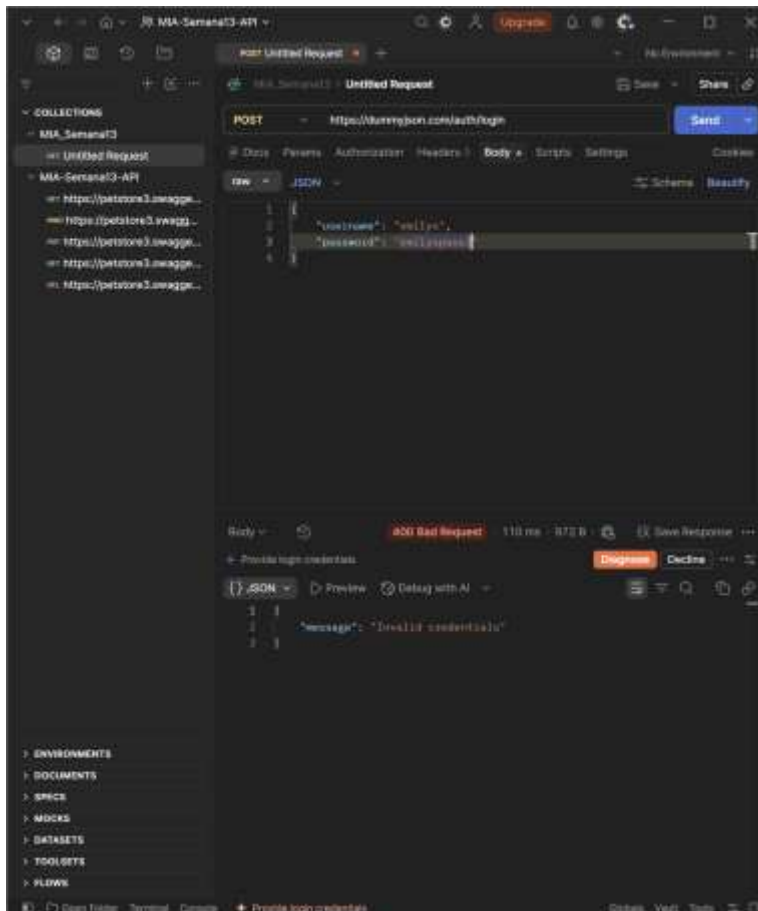
d) Utilice el endpoint: <https://dummyjson.com/auth/login>

e) En Body seleccione raw y JSON.

Ingrese las credenciales de prueba: Username: emilys | Password: emilyspass.

`{ "username": "emilys", "password": "emilyspass" }`

f) Ejecute la solicitud y localice en la respuesta el campo accessToken.



2. Utilizar token

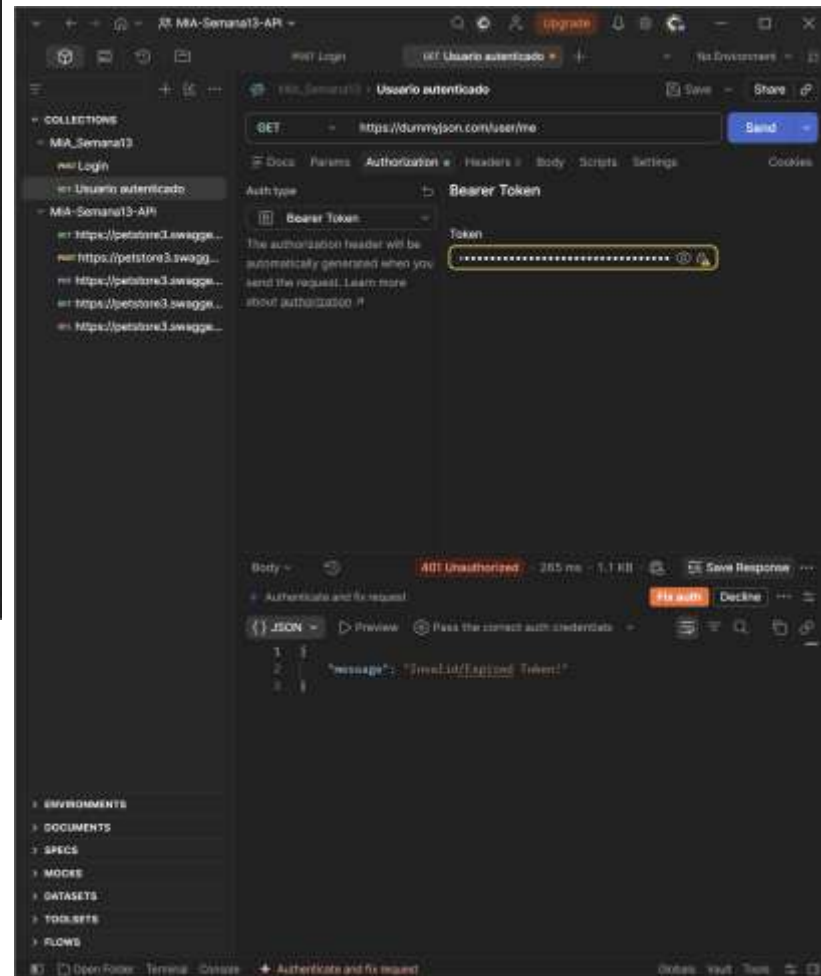
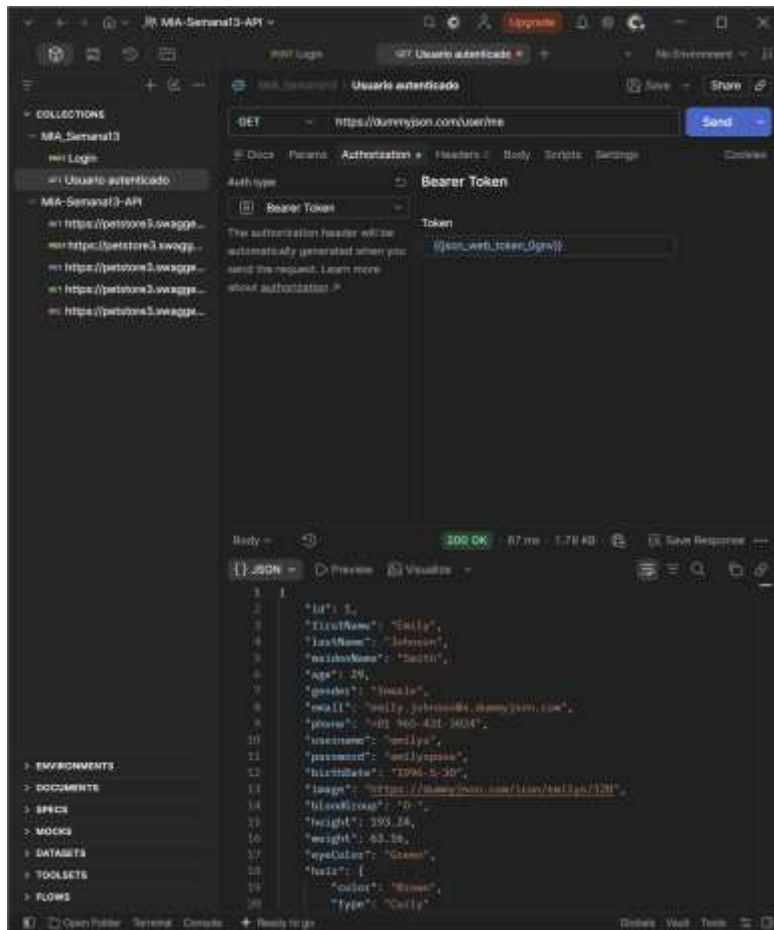
a) Cree una segunda solicitud llamada: Usuario autenticado.

b) Seleccione GET.

c) Utilice: <https://dummyjson.com/user/me>

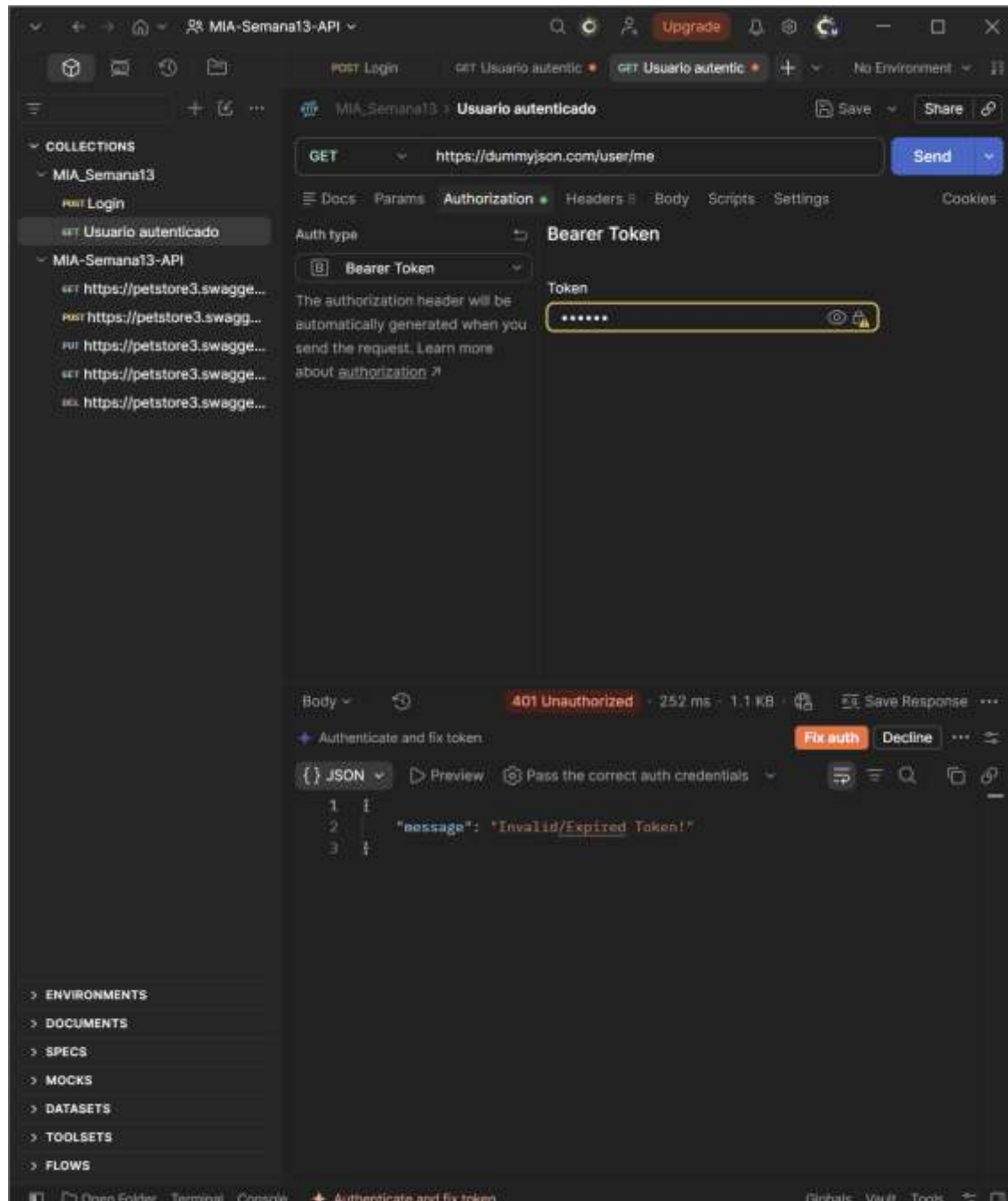
d) En Authorization seleccione Bearer Token.

- e) Copie el accessToken obtenido en la solicitud Login.
- f) Ejecute la solicitud.
- g) Verifique que la API devuelve la información del usuario autenticado.



3. Comprobar qué ocurre sin autenticación

- Duplique la solicitud Usuario autenticado.
- Quite el token.
- Ejecute nuevamente la solicitud.
- Compare la respuesta con la solicitud autenticada.



- Registre el código HTTP y explique la diferencia.

Ahora me dice que no tengo autorización, porque puse un token invalido. **ERROR 401**

f) Bearer Token es una cadena de caracteres generada por un servidor que otorga acceso a recursos protegidos a cualquier persona o sistema que lo posea, sin necesidad de presentar otro tipo de credencial (como un usuario o contraseña) en cada petición.

Parte 2 – Proteger la Connection String de Azure Storage

Utilice como base el programa C# desarrollado anteriormente para conectarse a Azure Storage. No se requiere crear nuevamente el Storage Account

5. Localizar la Connection String

- Abra el proyecto C# de Azure Storage.
 - En el Explorador de soluciones, abra el archivo donde actualmente está escrita la Connection String. En un proyecto sencillo normalmente será Program.cs.
 - Busque la variable que contiene la Connection String.
 - Identifique el valor que comienza con información como DefaultEndpointsProtocol=...
-

6. Reemplazar la Connection String

Reemplace el valor escrito directamente en el programa por una lectura de una variable de entorno llamada:

AZURE_STORAGE_CONNECTION_STRING

La idea es que el programa siga utilizando la misma variable para conectarse a Azure, pero ahora el valor será obtenido desde el entorno y no desde el código

7. Crear la variable para ejecutar localmente

- En la carpeta principal del proyecto, cree un archivo llamado .env.
 - Dentro del archivo agregue la variable:
AZURE_STORAGE_CONNECTION_STRING=TU_CONNECTION_STRING
 - Reemplace SU_CONNECTION_STRING por la Connection String de su Storage Account.
 - Verifique que el programa lea la variable desde el .env y ejecute nuevamente el programa.
 - Compruebe que el programa continúa funcionando correctamente.
 - No publique el archivo .env en GitHub**, ya que contiene la credencial real.
-

8. Crear el Secret en Github

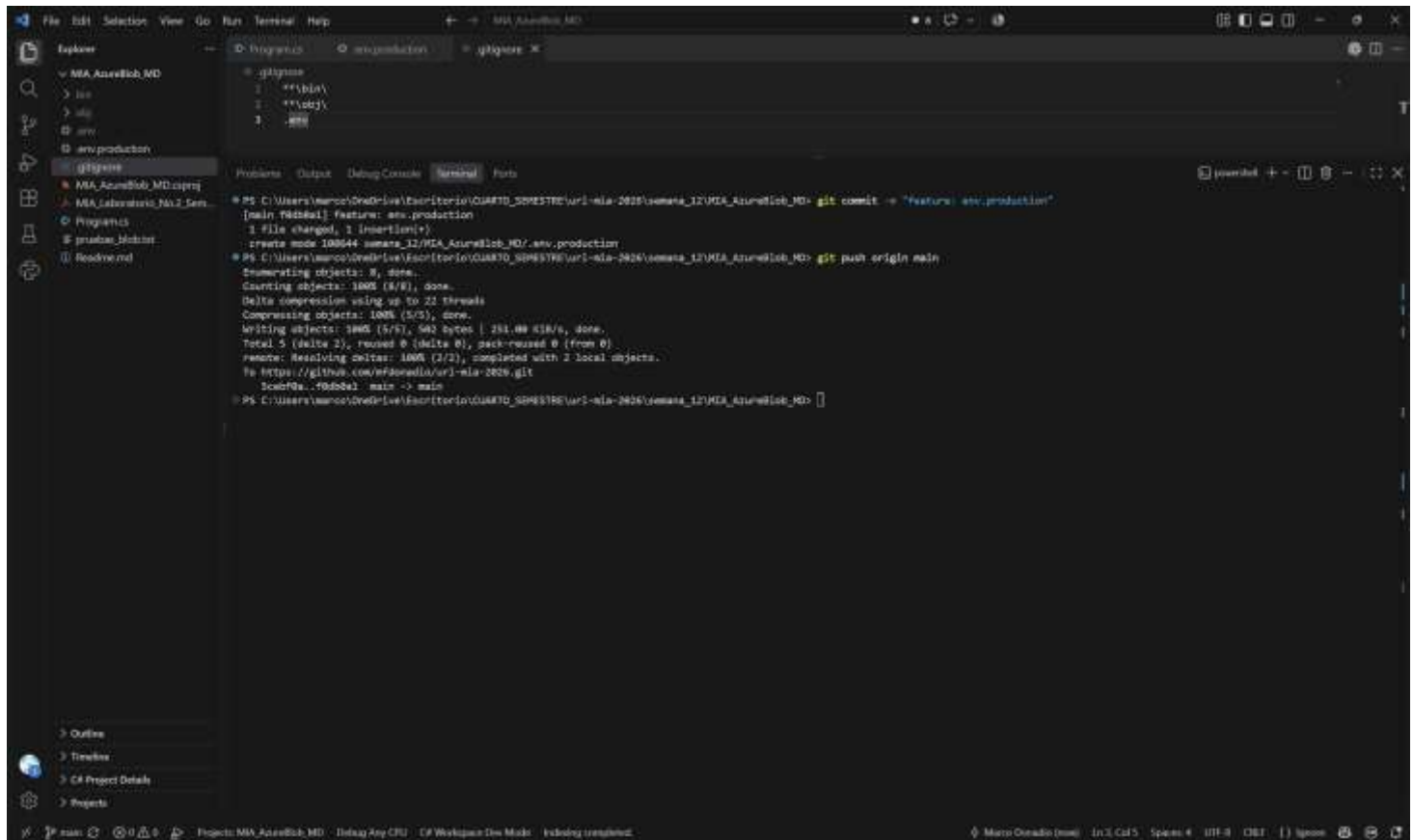
- Ingresa al repositorio de su proyecto en GitHub.
- Seleccione Settings.
- En el menú lateral seleccione Secrets and variables.
- Seleccione Actions.
- Seleccione New repository secret.
- En Name escriba: **AZURE_STORAGE_CONNECTION_STRING**
- En Secret coloque la Connection String de Azure Storage.
- Seleccione Add secret.

Entrega

Dentro del repositorio de GitHub, cree la carpeta semana-13/lab2/ y coloque en ella un PDF con la solución del laboratorio.

Incluya también los archivos de respaldo generados durante la práctica, así como los pantallazos que evidencien las

pruebas realizadas.
Enviar link de github a plataforma de la universidad.



The screenshot shows a Visual Studio Code interface with a terminal window open. The terminal displays the following commands and output:

```
PS C:\Users\marco\OneDrive\Escritorio\QUARTO_SOPRESTRE\url-mia-2026\semana_12\MEA_AzureBlob_MD> git commit -m "feature: env.production"
[main f0db8d1] feature: env.production
1 file changed, 1 insertion(+)
```

The output indicates that a commit was successfully created on the main branch. The commit message is "feature: env.production". The terminal also shows the output of the git push command, which successfully pushed the commit to the origin main branch.

```
PS C:\Users\marco\OneDrive\Escritorio\QUARTO_SOPRESTRE\url-mia-2026\semana_12\MEA_AzureBlob_MD> git push origin main
Enumerating objects: 8, done.
Counting objects: 3005 (8/8), done.
Delta compression using up to 22 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (5/5), 562 bytes | 251.00 KiB/s, done.
Total 5 (delta 2), reused 0 (delta 0), pack-reused 0 (from 0)
remote: Receiving deltas: 100% (2/2), completed with 2 local objects.
To https://github.com/mfalconia/url-mia-2026.git
Scm7f0a..f0db8d1 main -> main
```

The terminal output shows that the push was successful, and the commit was pushed to the origin main branch. The commit hash is f0db8d1.